

## 3.6 with Statement (Context Manager)

with بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 15 of 17

★ Golden

### 3.6 with Statement (Context Manager)

★ Golden

#### LEARN IT

It is easy to forget `.close()`. If the program crashes before `.close()` runs, the file may be incomplete or corrupted. Python's `with` statement is a **context manager**: it opens the file, runs the indented block, and **always closes** the file when the block ends — even if an error occurs inside.

with open(...) as f → use f → block ends → file closed automatically

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")
# no .close() here — with already closed the file
```

After the indented block, `new_file` is closed. You do not write `new_file.close()`.

### 3.7 Activity: reading file content

`.read()` pulls the whole file into RAM. For a huge file that can waste memory. The professional pattern is to **iterate line by line**.

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")

with open('myfile.txt', 'r') as read_file:
    for line in read_file:
        print(line, end="")
print("\n\nFile read successfully.")
```

#### OUTPUT

Context managers are safe!

File read successfully.

`end=""` stops `print` from adding a second newline: each line from the file already ends with `\n`.

#### WORKED EXAMPLE

**Three ways to read** the same small file.

```

with open('sample.txt', 'w') as f:
    f.write("Ali 80\nSara 90\nAyesha 75\n")

with open('sample.txt', 'r') as f:
    print("=== .read() ===")
    print(repr(f.read()))

with open('sample.txt', 'r') as f:
    print("=== .readlines() ===")
    print(f.readlines())

with open('sample.txt', 'r') as f:
    print("=== line by line ===")
    for line in f:
        print(line, end="")

```

#### OUTPUT

```

=== .read() ===
'Ali 80\nSara 90\nAyesha 75\n'
=== .readlines() ===
['Ali 80\n', 'Sara 90\n', 'Ayesha 75\n']
=== line by line ===
Ali 80
Sara 90
Ayesha 75

```

`repr` shows the hidden `\n` characters. `.readlines()` is handy when you need a list (the grade tracker uses this). Line-by-line looping is best for large files because only one line sits in memory at a time.

#### REMEMBER IT

**with = "take, use, put back."** Prefer `with open(...) as f:` in every new program. You still choose the mode: `'r'`, `'w'` or `'a'`.

#### COMMON MISTAKE

You cannot read from `f` after the `with` block — the file is already closed. Do all reading and writing inside the block. Also, `print(line)` without `end=""` looks double-spaced because both the line and `print` add a newline.

#### اردو میں سمجھیں

`with` بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔ بڑی فائلیں ایک ایک سطر کے لوپ سے پڑھنا بہتر ہے بجائے پورے مواد کو `.read()` سے میموری میں بھرنے کے۔  
`print(line, end="")` دگنی خالی سطر روکتا ہے۔

#### EXAM TIP

Define the `with` statement as a context manager that guarantees close. Contrast it with manual `open / close`. Be ready to write a four-line program that writes with `with` and a four-line program that reads line by line. Mention memory: iterating is safer for large files than `.read()`.

#### CHECK YOURSELF

1. What does a context manager guarantee for a file?
2. Why pass `end=""` when printing each line from a file?

---

Answers: 1. That the file is closed when the with block ends, even if an error occurs. 2. Because each line already ends with `\n`; print would otherwise add another newline and the output would look double-spaced.

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

**MCQ 1.** with open(...) as f:

- a. Closes the file automatically, even after errors
- b. Never closes
- c. Deletes the path
- d. Converts to pandas

**MCQ 2.** The with-block is a:

- a. Context manager
- b. Thread
- c. Socket always
- d. K-map

**MCQ 3.** If an exception happens inside with:

- a. The file still closes
- b. The file stays locked forever
- c. Python uninstalls
- d. The disk formats

**MCQ 4.** You still choose the mode:

- a. r/w/a as before
- b. with replaces modes
- c. Modes are only for SQL
- d. Modes are HCI

**MCQ 5.** This is ★ because it is the:

- a. Safe modern pattern for files
- b. Old optional style
- c. Only for pandas
- d. Only for Figma

**MCQ 6.** f is only valid:

- a. Inside the with block
- b. After the block always
- c. Before open
- d. In another file automatically

## Extra short questions

1. Write the canonical read pattern.
2. Why it beats try/finally for beginners.
3. Can you nest with for two files?
4. What if the path is wrong in with-open r?

## Extra long question

1. Rewrite a grade-tracker snippet from open/close to with, and show what happens if int() fails on a line. Why is with safer? ★ Golden.

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. with open("a.txt") as f: data=f.read()
2. Less boilerplate; close is guaranteed.
3. Yes — with open(a) as f, open(b) as g:
4. Still FileNotFoundError — with does not create the file in r.

### Long

1. Exception in parse still closes. Compare to forgotten close on the error path. Sample file 3 lines, one bad.